

Progetto di Apprendimento Automatico

Regressione, Classificazione e Clustering: implementazioni da zero

Matteo Barbieri

Apprendimento Automatico

30 giugno 2026

1 Regressione

2 Classificazione

3 Clustering

1 Regressione

2 Classificazione

3 Clustering

Che cos'è la regressione

Apprendimento supervisionato a target *continuo*

Dati m esempi $(x^{(i)}, y^{(i)})$, vogliamo una funzione $h_{\theta}(x) \approx y$ che predica un valore **continuo** e **numerico**.

Caso di studio: Combined Cycle Power Plant (CCPP)

- 9568 misurazioni di una centrale elettrica.
- $n = 4$ feature: temperatura (AT), vuoto (V), pressione (AP), umidità (RH).
- Target: potenza erogata PE (MW).

$$PE \approx \theta_0 + \theta_1 AT + \theta_2 V + \theta_3 AP + \theta_4 RH$$

Aggiungiamo la feature fittizia $x_0 = 1$ (bias trick) per l'intercetta θ_0 , in modo da poter generalizzare meglio le formule:

$$h_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x} = \sum_{j=0}^n \theta_j x_j, \quad \theta \in \mathbb{R}^{n+1}.$$

- m = numero di esempi, n = numero di feature.
- θ_0 = intercetta; $\theta_1, \dots, \theta_n$ = pesi delle feature.
- L'ipotesi è **lineare nei parametri**: un semplice prodotto scalare.

Misuriamo l'errore con l'**errore quadratico medio** (con il fattore $\frac{1}{2}$):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

- Il quadrato penalizza molto gli errori grandi.
- Il fattore $\frac{1}{2}$ semplifica la derivata (scompare il 2).
- $J(\theta)$ è **convessa** $\Rightarrow$ un **unico minimo globale**.

Conseguenza

Qualunque metodo corretto converge *agli stessi parametri θ ottimi*.

Gradient Descent (Batch)

Minimizziamo $J(\theta)$ aggiornando i parametri nella direzione opposta al gradiente.

Regola di aggiornamento — ripeti fino a convergenza, *simultaneamente* per ogni $j = 0, 1, \dots, n$:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- $(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) = \mathbf{errore}$ sull'esempio i .
- Ogni θ_j si corregge in base all'errore medio, pesato dalla sua feature x_j .
- $\alpha > 0 = \mathbf{learning\ rate}$ (ampiezza del passo).
- Stop quando $|J_{t-1} - J_t| < \varepsilon$.

Per far convergere bene il gradient descent conviene riportare le feature alla stessa scala (pressione ~ 1000 mbar vs umidità in %):

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

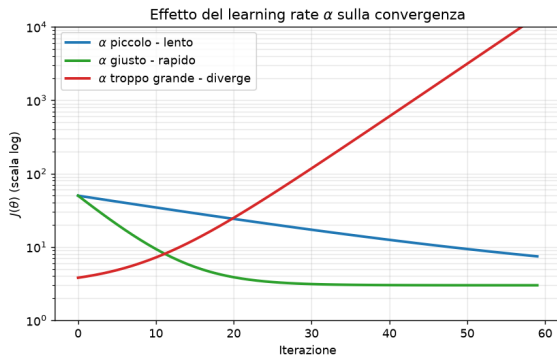
- Senza scaling gli attributi con domini più ampi dominano il modello.
- Con lo scaling la discesa del gradiente è **dritta e veloce**.
- μ_j, σ_j stimati **solo sul training set** (niente data leakage).

Il ruolo del learning rate α

- α **piccolo** $\rightarrow$ convergenza lenta.
- α **giusto** $\rightarrow$ discesa rapida e stabile.
- α **troppo grande** $\rightarrow$ **divergenza**.

Perché diverge

Con un passo troppo grande il GD **scavalca** il minimo e finisce sull'altro lato della valle, *più in alto* di prima. Ripetendo, le oscillazioni si **amplificano** invece di smorzarsi.



Equazione normale: la soluzione in forma chiusa

Essendo J convessa (in quanto quadrato di una retta), possiamo annullare il gradiente direttamente:

$$\theta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Riferimento **esatto** per validare il GD (confronto nel notebook).

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_i (\hat{y}_i - y_i)^2}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- RMSE nell'unità del target (MW).
- RMSE penalizza gli outlier.
- R^2 : frazione di varianza spiegata (1=perfetto, 0=media).

Cross-validation k -fold

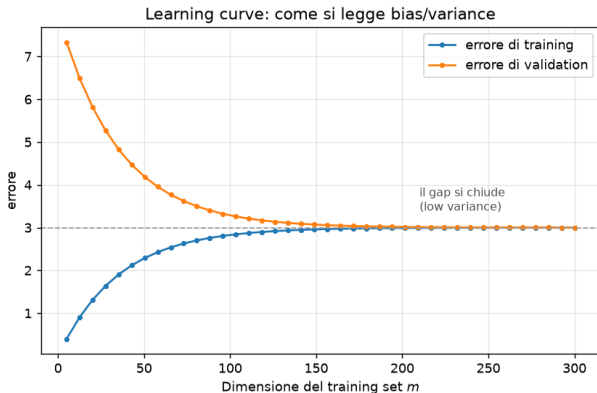
- Si divide il dataset in k fold; a turno uno è validation, gli altri training.
- Ogni esempio finisce in validation **esattamente una volta**.
- Stima dell'errore di generalizzazione **più robusta** di un singolo split.
- Lo scaler viene ri-adattato *dentro ogni fold* (no leakage).

Nel progetto: $k = 5$, riportiamo media $\pm$ deviazione standard.

Learning curve: diagnosi bias/variance

Come si costruisce: si addestra il modello *più volte* con quote **crescenti** di training (m crescente), misurando ogni volta l'errore su train e validation. Non serve un singolo numero, ma la *tendenza* delle due curve:

- Curve che convergono a un errore **basso** → **low bias**: il modello non fatica ad adattarsi.
- Gap che si **chiude** e resta minimo → **low variance** (no overfitting): il modello non si specializza troppo sul train.
- Curve **piatte** ad errore alto → più dati non aiutano (limite del modello: bias).

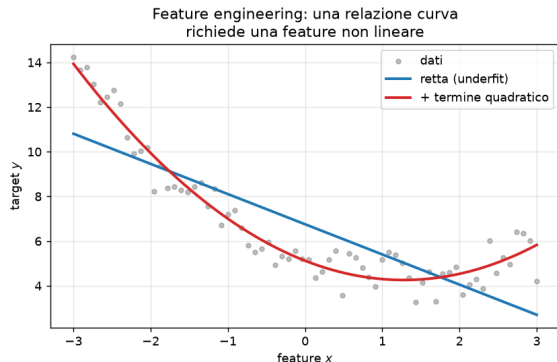


Feature engineering: ridurre il bias

L'errore residuo di un modello lineare è in parte **bias**: la relazione vera può non essere una retta. Si aggiungono **feature non lineari** (es. x^2) — il modello resta **lineare nei parametri**.

Quando conviene:

- 1 Su una feature **dominante** (peso grande).
- 2 Che **mostra curvatura**: la retta sotto/sovrastima agli estremi (figura).



Regolarizzazione: serve qui?

Per contrastare l'*overfitting* si aggiunge una penalità sui pesi (Ridge / L2):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

(l'intercetta θ_0 non si regolarizza)

Quando serve (e quando no)

- Utile in presenza di **overfitting**: tanti parametri rispetto agli esempi.
- Se $m \gg n$ e la learning curve non mostra varianza, $\lambda = 0$: regolarizzare introdurrebbe solo **bias** senza guadagno.

Scelta di λ guidata dai dati (validation / CV) — vedi notebook.

1 Regressione

2 Classificazione

3 Clustering

Apprendimento supervisionato a target *discreto*

Dati m esempi $(\mathbf{x}^{(i)}, y^{(i)})$, vogliamo predire un'**etichetta di classe** $y \in \{c_1, \dots, c_k\}$ (valore **categorico**, non numerico).

Caso di studio: Breast Cancer Wisconsin (WDBC)

- 569 biopsie, $n = 30$ feature numeriche (raggio, texture, area, ...).
- Problema **binario**: $y \in \{\text{maligno}, \text{benigno}\}$ (212 vs 357).
- Obiettivo: dato il profilo di una cellula, predirne la diagnosi.

Due famiglie di classificatori probabilistici

Discriminativi

Modellano *direttamente* il confine: $P(c \mid x)$.

- es. regressione logistica, reti neurali.
- “dove passa la frontiera fra le classi?”

Generativi

Modellano *come sono fatti i dati* di ogni classe: $P(x \mid c)$ e $P(c)$.

- es. **Naïve Bayes**.
- “che aspetto ha una cellula maligna?”

Naïve Bayes è un modello **generativo**: impara un modello probabilistico *per ciascuna classe* e poi usa Bayes per classificare.

Regola di Bayes e criterio MAP

Per classificare x vogliamo la classe *a posteriori* più probabile:

$$P(c | x) = \frac{\overbrace{P(c)}^{\text{prior}} \overbrace{P(x | c)}^{\text{likelihood}}}{\underbrace{P(x)}_{\text{evidenza}}}$$

Regola MAP (*Maximum A Posteriori*)

Assegna x alla classe con $P(c | x)$ **più grande**; equivalentemente

$$P(c | x) \propto P(c) P(x | c)$$

- L'evidenza $P(x)$ è **uguale per tutte le classi** $\Rightarrow$ si può ignorare nel confronto.
- Restano da stimare dai dati solo **prior** $P(c)$ e **likelihood** $P(x | c)$.

L'assunzione "naïve"

Stimare la likelihood **congiunta** $P(x | c) = P(x_1, \dots, x_n | c)$ è in generale **impraticabile**: troppe combinazioni di feature.

Ipotesi di indipendenza condizionata

Data la classe, le feature sono **indipendenti tra loro**:

$$P(x | c) = \prod_{j=1}^n P(x_j | c)$$

- Una congiunta in n dimensioni $\rightarrow n$ distribuzioni **univariate** (facili da stimare).
- Sostituendo nella regola MAP: $P(c | x) \propto P(c) \prod_{j=1}^n P(x_j | c)$.

Feature continue: Naïve Bayes *gaussiano*

Le feature di WDBC sono **continue** $\Rightarrow$ per ogni feature e ogni classe modelliamo $P(x_j | c)$ con una **gaussiana**:

$$P(x_j | c) = \mathcal{N}(x_j; \mu_{jc}, \sigma_{jc}^2) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right)$$

- Parametri stimati a **massima verosimiglianza**, **solo dagli esempi di quella classe**:

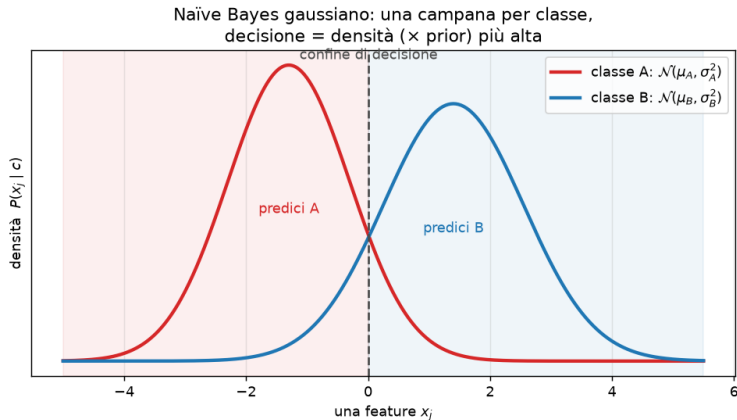
$$\mu_{jc} = \frac{1}{m_c} \sum_{i: y^{(i)}=c} x_j^{(i)}, \quad \sigma_{jc}^2 = \frac{1}{m_c} \sum_{i: y^{(i)}=c} (x_j^{(i)} - \mu_{jc})^2$$

- Ogni classe diventa un insieme di n “campane”: un **modello generativo** della classe.

$\mathcal{N}$ è una *densità*, non una probabilità: nel confronto fra classi non crea problemi, perché conta solo *quale* densità è più alta.

Visualizzazione: una gaussiana per classe

- Ogni classe è una **campana** (qui su una sola feature).
- Si predice la classe con **densità $\times$ prior più alta**.
- Dove le campane si incrociano nasce il **confine di decisione**.



Apprendimento (fit)

- $P(c) = \frac{m_c}{m}$ (frequenza della classe).
- μ_{jc}, σ_{jc}^2 per ogni classe e feature.

Soluzione in forma chiusa: una sola passata sui dati, niente gradient descent, niente learning rate.

Predizione (predict)

Si sceglie la classe col **punteggio più grande**; in spazio **logaritmico**:

$$\text{punteggio}(c) = \log P(c) + \sum_{j=1}^n \log P(x_j | c)$$

Nota: niente standardizzazione

A differenza della regressione, NB è **invariante alla scala**: ogni feature ha la sua gaussiana per classe, quindi riscalarla non cambia le predizioni.

Come si classifica un esempio

Per ogni classe sommiamo prior e log-verosimiglianze; vince il **punteggio più grande**:

(esempio illustrativo, numeri inventati)

termine	classe A	classe B
$\log P(c)$ (prior)	-0.7	-0.5
feature vicina alla media di A	+2.0	+0.5
feature nella coda di B	-1.0	-9.0
$\vdots$ (altre feature)	$\vdots$	$\vdots$
punteggio totale	+5	-22

- Una feature il cui valore cade nella **coda** della gaussiana di una classe dà $\log \mathcal{N}$ molto negativo $\Rightarrow$ ne abbassa drasticamente il punteggio.
- Vince la classe il cui modello generativo **spiega meglio** l'esempio.

Il calcolo su una biopsia reale di WDBC è nella demo del notebook.

L'assunzione naïve: perché regge

Le feature di WDBC sono fortemente correlate

radius, perimeter, area misurano quasi la stessa cosa: l'indipendenza condizionata è quindi falsa.

- Le feature ridondanti vengono **contate più volte** $\Rightarrow$ posterior **sovra-sicuri** (vicini a 0 o 1).
- **Ma la decisione tiene**: per classificare serve solo che la classe giusta abbia il **punteggio più alto**, non una probabilità calibrata.
- La **decisione MAP** (classe più probabile) è robusta a stime distorte $\Rightarrow$ accuracy alta nonostante l'assunzione violata.

Naïve Bayes è un buon *classificatore* anche quando è un cattivo *stimatore di probabilità*.

Valutare un classificatore: matrice di confusione

reale	predetto	
	maligno	benigno
maligno	TP	FN
benigno	FP	TN

Positivo = **maligno** (la classe che vogliamo individuare).

$$\text{Accuracy} = \frac{TP + TN}{m}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

Perché non basta l'accuracy

- Su classi **sbilanciate** l'accuracy inganna: predire “sempre benigno” darebbe già $\sim 63\%$ di accuratezza senza imparare nulla.
- In ambito **medico** l'errore grave è il **falso negativo** (un tumore maligno scambiato per benigno).
- **Recall** (sensibilità) sul maligno = quota di tumori effettivamente individuati $\Rightarrow$ la metrica da tenere d'occhio qui.
- F_1 riassume precision e recall in un solo numero (media armonica).

Validazione robusta con **5-fold cross-validation** (come per la regressione).

1 Regressione

2 Classificazione

3 Clustering

Che cos'è il clustering

Apprendimento *non* supervisionato

Dati m esempi $x^{(1)}, \dots, x^{(m)}$ **senza etichette**, vogliamo raggrupparli in K **cluster** di punti simili tra loro.

Caso di studio: WDBC senza la diagnosi

- Usiamo le 569 biopsie **ignorando** l'etichetta maligno/benigno.
- Cerchiamo una struttura nei dati con $K = 2$ cluster.
- *A posteriori* confrontiamo i cluster con la diagnosi reale: il modello ritrova i due gruppi senza averli mai visti?

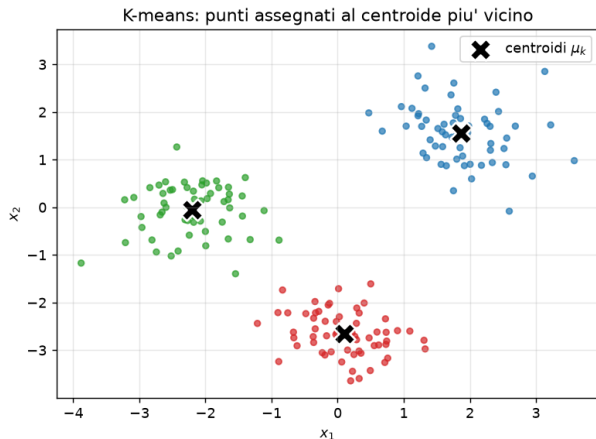
L'algoritmo k-means

Si parte da K centroidi e si ripete fino a convergenza:

- 1 **Assegnazione:** $c^{(i)} :=$ indice del centroide μ_k **più vicino** a $x^{(i)}$ (distanza $\|x^{(i)} - \mu_k\|$ minima)
- 2 **Aggiornamento:** ogni centroide diventa la media dei suoi punti

$$\mu_k := \text{media dei punti del cluster } k$$

Due passi alternati, semplici e senza iperparametri di ottimizzazione.



Obiettivo: la distorsione

K-means minimizza la **distorsione** (distanza media dei punti dal proprio centroide):

$$J = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c(i)}\|^2$$

- I due passi (assegnazione, aggiornamento) **non aumentano mai** $J \Rightarrow$ l'algoritmo converge.
- Converte però a un minimo **locale**: il risultato dipende dai centroidi iniziali.

La standardizzazione qui è essenziale

K-means usa la **distanza euclidea**: senza scaling le feature con valori grandi dominerebbero. (Al contrario del Naïve Bayes, invariante alla scala.)

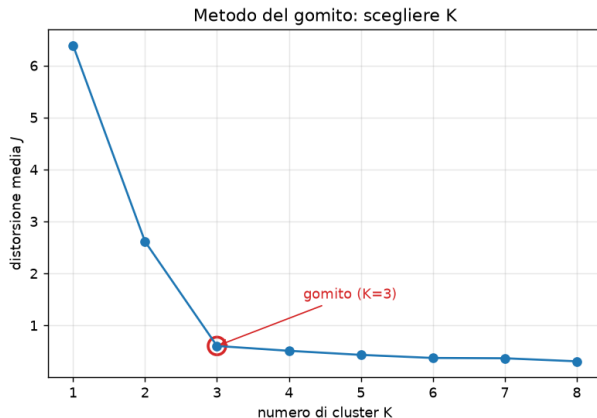
- Centroidi iniziali **casuali** $\Rightarrow$ rischio di finire in un **minimo locale** scadente.
- **Multi-restart**: si ripete k-means da più inizializzazioni e si tiene quella con **distorsione J minima**.
- **k-means++**: sceglie centroidi iniziali *ben distanziati* (ogni nuovo centro con probabilità $\propto D(x)^2$), partenza più robusta.

Nel progetto: `n_init= 10 restart, init k-means++`.

Scegliere il numero di cluster K

Metodo del gomito: si traccia la distorsione J al crescere di K .

- J cala sempre all'aumentare di K (più cluster $\rightarrow$ punti più vicini ai centroidi).
- Il “**gomito**” è il punto oltre il quale il guadagno diventa marginale.
- Si sceglie K al gomito: buon compromesso fra compattezza e semplicità.



Validazione esterna: purity

Se disponiamo di **classi vere** (qui la diagnosi), confrontiamo i cluster con esse.

Purity

$$P = \frac{1}{N} \sum_{k \in K} \max_{d \in D} |k \cap d|$$

Ogni cluster viene assegnato alla **classe di maggioranza**; la purity è la frazione di punti “corretti”. Vale 1 se ogni cluster contiene una sola classe.

- Si costruisce la **matrice di confusione** classi-vs-cluster.
- È una misura *esterna*: richiede le etichette, usate solo per valutare.

Validazione interna: silhouette

Senza etichette, misura quanto i cluster sono **compatti e separati**. Per ogni punto:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $a(i)$: distanza media *dentro* il cluster.
- $b(i)$: distanza media dal cluster *vicino* più prossimo.
- $s \rightarrow 1$ ottimo, $s \rightarrow 0$ cluster sovrapposti.

